

Library Management System

COEP Technological University

14 June 2024

Team 4 (Atharva, Pratik , Saurav)

Library Management System Project Report

Table of Contents

1. Introduction

2. Project Overview

3. Features and Functionality

4. Code Explanation

- Classes
- Functions and Methods

5. User Interfaces

- Initial Menu
- Admin Menu
- Patron Menu

6. File Operations

7. Security Measures

8. Future Enhancements

9. Conclusion

Introduction

The Library Management System is a software application designed to facilitate the management of books and patrons within a library. It provides functionalities for managing book inventories, patron records, borrowing, and returning books, and handling overdue fines. The system also supports basic authentication and role-based access control for patrons and administrators.

Project Overview

This system is built using Python and utilizes CSV files for data storage. It includes core functionalities like adding, viewing, searching, and deleting books and patrons, as well as borrowing and returning books. It also manages overdue books and applies fines where applicable.

Features and Functionality

Books:

- Add, view, search, delete books.
- Borrow and return books.
- Manage book reviews.

Patrons:

- Register and manage patrons.
- Login and logout functionality.
- Role-based access (Admin/Patron).

Administration:

- Manage book inventory.
- View and manage patron records.
- Check and handle overdue books.

Code Explanation

Classes

Book

Purpose: The Book class represents a book in a library or inventory system, capturing essential details about the book such as its ID, title, author, quantity, cost, due date, and reviews

- Attributes: book_id, title, author, quantity, Cost, due_date, reviews.

```
class Book:                                # Class: A template or blueprint for creating objects
    def __init__(self, book_id, title, author, quantity, Cost, due_date=None,):    # Book is
a class that defines attributes like title, author, and pages
        self.book_id = book_id # self.book_id refers to the instance's attribute
        self.title = title
        self.author = author
        self.quantity = quantity
        self.Cost = Cost
        self.due_date = due_date
        self.reviews = ()           # store reviews
```

This class provides a template for creating book objects with specified attributes and default values for some of them. It includes attributes for managing inventory details and potential borrowing features, indicated by the due_date and reviews.

- Methods: add_review, display_reviews.

Patron

Purpose: The Patron class represents a user (patron) in a library, capturing essential details such as the user's ID, name, email, password, and role.

```
class Patron:
    def __init__(self, patron_id, name, email, password, role='patron',):
        self.patron_id = patron_id
        self.name = name
        self.email = email
        self.password = password
        self.role = role
```

This class provides a template for creating Patron objects with attributes for identification (Patron_Id), personal details (Name and email), authentication (password), and access control (role).

- Attributes: patron_id, name, email, password, role.

Library

Purpose: The library class manages a collection of books, patrons, and the borrowing status of books, providing methods for loading and managing these entities.

- Attributes: books, patrons, borrowed_books, current_patron.

```
class Library:
    def __init__(self):
        self.books = []
        self.patrons = []
        self.borrowed_books = {}
        self.current_patron = None
        self.load_books()
        self.load_patrons()
        self.load_borrowed_books()
```

The methods load_books, load_patrons, and load_borrowed_books are placeholders for functionality that loads initial data into the system. These methods likely handle data retrieval from the CSV to populate the library's records upon initialization.

- Methods: Various methods for loading/saving data, adding/viewing/deleting books and patrons, borrowing/returning books, handling overdue books, etc.

Functions and Methods

- **Data Management:** Methods like load_books, save_books, load_patrons, save_patrons, load_borrowed_books, save_borrowed_books handle file operations for persistent storage of books and patrons.
- **Operations:** add_book, delete_book, view_books, search_books_by_title manage book-related operations.
- **Patron Management:** add_patron, view_patrons, delete_patron, register_patron, login_patron, logout_patron handle patron management and authentication.
- **Borrow/Return Books:** borrow_book, return_book manage borrowing and returning of books.
- **Overdue Management:** check_overdue_books handles overdue books and applies fines ie rs100/- per day.

User Interfaces

Initial Menu

Provides options to register or log in:

```
Library Management System Menu:
1. Register Patron/Admin
2. Login
3. Exit
Enter your choice: █
```

- 1. Register Patron/Admin
- 2. Login
- 3. Exit

Admin Menu

Available to logged-in admins:

```
Welcome, A!

Admin Menu:
1. Add Book
2. View Books
3. Search a Book
4. Delet Book
5. Add Patron
6. View Patrons
7. Delet patron
8. Check overdue books
9. Logout
10. Exit
Enter your choice: █
```

- 1. Add Book
- 2. View Books
- 3. Search a Book
- 4. Delete Book
- 5. Add Patron
- 6. View Patrons
- 7. Delete Patron
- 8. Check Overdue Books
- 9. Logout
- 10. Exit

Patron Menu

Available to logged-in patrons:

```
Patron Menu:
1. View Books
2. search a book
3. Borrow Book
4. Return Book
5. Review Book
6. Check Overdue Books
7. Logout
8. Exit
Enter your choice: █
```

- 1. View Books
- 2. Search a Book
- 3. Borrow Book
- 4. Return Book
- 5. Review Book
- 6. Check Overdue Books
- 7. Logout
- 8. Exit

File Operations

The system uses CSV files for data persistence:

CSV files were prepared using chatgpt in order to include the specific columns.

1. **books.csv**: Stores book details.

https://github.com/NothingIssue/SDW_project/blob/58485b56c841a2db1e2aef380baacf1727b1a834/books.csv

- - Fields: book_id, title, author, quantity, Cost, reviews, due_date.

2. **patrons.csv**: Stores patron details.

https://github.com/NothingIssue/SDW_project/blob/58485b56c841a2db1e2aef380baacf1727b1a834/patrons.csv

- - Fields: patron_id, name, email, password, role.

3. **borrowed_books.csv**: Stores borrowing records

https://github.com/NothingIssue/SDW_project/blob/58485b56c841a2db1e2aef380baacf1727b1a834/borrowed_books.csv

- - Fields: patron_id, book_id, due_date.

The system loads data from these files during initialization and saves changes upon performing operations.

Complete Code

https://github.com/NothingIssue/SDW_project/blob/58485b56c841a2db1e2aef380baacf1727b1a834/trial%20test1.py

Security Measures

- Password Hashing: Passwords are hashed using SHA-256 before storage to ensure security.

```
hash_password = hash.sha256(password.encode()).hexdigest() # converted into hash
```

- Role-Based Access Control: Separate menus for patrons and admins to restrict access based on roles.

Future Enhancements

- Enhanced User Interface: Implement a graphical user interface (GUI) for a better user experience.
- Email verification function.
- Password change option.
- Borrow book limit per patron.
- In search a book option different genres can be added.
- Searching a book using ID or words used in the title.
- Fine Calculation: Implement a more sophisticated fine calculation and notification system.

Conclusion

Developing the Library Management System has been both challenging and enlightening. This project has significantly broadened our knowledge and skills in several key areas.

We encountered and overcame various challenges during the development process:

- **Password Storage in Hash Form:** Implementing secure password storage using hashing mechanisms was crucial for ensuring user security. This challenge deepened our

understanding of encryption techniques and best practices in safeguarding sensitive information.

- **Upper and Lower Case Management in Searching:** Handling case sensitivity in searching books and patron records within CSV files required us to refine our data processing techniques. This task improved our ability to manipulate and search through data efficiently, ensuring accurate and user-friendly search functionality.
- **Compact and Efficient Code:** Writing compact and efficient code was a key goal, for which we leveraged AI tools. This experience not only streamlined our coding practices but also introduced us to advanced code optimization strategies and the benefits of AI assistance in software development.

Through addressing these challenges, we enhanced our understanding of various functions, improved our code management practices, and became proficient in using CSV files effectively. We also learned to properly link and organize menu systems, which contributed to a more cohesive and intuitive user interface.

Overall, the Library Management System project has enriched our technical knowledge, improved our problem-solving abilities, and provided valuable insights into best practices for software development. This experience has equipped us with a solid foundation for future projects and innovations in library management systems.

Machine Learning Project Report: **ChatGPT Reviews**

COEP Technological University

14 June 2024

Team 4 (Atharva, Pratik , Saurav)

1 Introduction

1.1 Background

The ChatGPT Android App is a widely-used application that leverages OpenAI's conversational AI, ChatGPT. With the app's popularity, user reviews have become a crucial source of feedback, offering insights into user satisfaction and areas for improvement. These reviews and ratings can be analyzed to understand user sentiment and predict future trends.

1.2 Problem Statement

This project focuses on analyzing the user reviews and ratings of the ChatGPT Android App. The primary objectives include visualizing the dataset, understanding the reasons for its creation, applying machine learning techniques to analyze the data, and reporting the results.

1.3 Objectives

- Visualize the dataset to understand distribution and trends.
- Explore the motivation behind the creation of the dataset.
- Apply machine learning techniques such as linear regression, logistic regression, and neural net-works.

- Report and interpret the results of the analysis.

1.4 Scope

The project encompasses data cleaning, visualization, and the application of machine learning techniques to analyze and predict user ratings. The insights gained can help in improving user experience and understanding user sentiment better.

2 Methodology

2.1 Data Collection

The dataset used in this project is obtained from Kaggle: ChatGPT Reviews - Daily Updated. It contains user reviews and ratings for the ChatGPT Android App, updated daily.

<https://www.kaggle.com/datasets/ashishkumarak/chatgpt-reviews-daily-updated>

2.2 Data Preprocessing

Data preprocessing involves several steps to clean and prepare the data for analysis:

- **Removing Duplicates:** Duplicate entries are removed to ensure the data is unique and accurate.
- **Handling Missing Values:** Missing values in critical columns are addressed. Rows with null content are removed, and missing app versions are imputed based on the most frequent values in corresponding score subgroups.
- **Dropping Irrelevant Columns:** Columns that do not contribute to the analysis, such as 'reviewId' and 'userName', are dropped.

Listing 1: Data Cleaning and Preprocessing

```
# Removing duplicates  
# This removes duplicate rows from the DataFrame.  
reviews.drop_duplicates(inplace = True)  
  
# Dropping irrelevant columns  
# Dropping reviewId, userName columns  
reviews.drop(['reviewId', 'userName'], axis = 1, inplace = True)
```

```
# Handling missing values
modes_score = reviews.groupby('score')['appVersion'].agg(lambda
x:x.mode().iloc[0]).to_dict()

# Imputing null values in appVersion based on the most frequent
values in corresponding score subgroups
reviews['appVersion'].fillna(reviews['score'].map(modes_score),
inplace = True)
```

2.3 Tools and Technologies

- **Pandas:** A powerful data manipulation and analysis library, used for handling and cleaning the dataset.
- **Matplotlib and Seaborn:** Libraries for data visualization, helping to create informative and attractive plots.
- **Scikit-learn:** A machine learning library providing tools for model training, evaluation, and pre-processing.
- **TensorFlow/Keras:** Libraries for building and training neural networks, enabling more complex models for better predictions.
- **WordCloud:** A library to create word clouds from text data, providing a visual representation of the most frequent words.
- **Numpy:** A fundamental package for numerical computations, supporting arrays and matrices.

2.4 Model Selection

Several models are selected to address different aspects of the problem:

- **Logistic Regression:** Used for classifying reviews into high and low scores.
- **Linear Regression:** Employed to predict the exact rating scores.
- **Neural Network:** A more sophisticated model used to predict ratings, taking advantage of its capability to capture non-linear relationships in the data.

2.5 Implementation

- **Logistic Regression:** Converts text data to numerical data using TF-IDF, then trains a logistic regression model to classify review scores.

- **Linear Regression:** Predicts the score of reviews using linear regression after converting text data to numerical format.

- **Neural Network:** Builds a neural network model with multiple layers to predict review scores, offering more complexity and accuracy.

Listing 2: Model Implementation

Logistic Regression implementation

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the data
scaler = StandardScaler(with_mean=False) # StandardScaler with mean=False to avoid centering sparse data
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train the logistic regression model
log_reg_model = LogisticRegression()
log_reg_model.fit(X_train, y_train)

# Make predictions
y_pred_log_reg = log_reg_model.predict(X_test)
```

1. **Data Splitting:** The dataset is divided into training and testing subsets. This allows us to train the model on one portion and evaluate its performance on another, unseen portion.
2. **Data Standardization:** The feature values are standardized, meaning they're scaled to have a mean of 0 and a standard deviation of 1. This preprocessing step ensures uniformity in the data, aiding model convergence.
3. **Model Training:** A logistic regression model is trained using the training data. Logistic regression is a classification algorithm commonly used for binary classification tasks.
4. **Prediction:** The trained model is used to make predictions on the testing data. These predictions are then used to evaluate the model's performance.

This pipeline sets up the data, trains the model, and assesses its predictive capabilities.

Linear Regression

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=52)

# Standardize the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train the model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train) #This trains the linear regression model
```

```
# Make predictions
y_pred_lr = lr_model.predict(X_test)

# Evaluate the model
mse_lr = mean_squared_error(y_test, y_pred_lr)
print(f'Linear Regression MSE: {mse_lr}')
```

1. **Data Splitting:** It splits the dataset x and corresponding labels y into training and testing sets using the `train_test_split` function. Here, 90% of the data is used for training (x_{train} , y_{train}), and 10% for testing (x_{test} , y_{test}). The `random_state` parameter ensures reproducibility.
2. **Data Standardization:** The feature values are standardized using the `StandardScaler`. This process ensures that all features have a mean of 0 and a standard deviation of 1. It's applied separately to both the training and testing data.
3. **Model Training:** A linear regression model (`lr_model`) is instantiated and trained using the training data (x_{train} , y_{train}) through the `fit` method. Linear regression attempts to find the linear relationship between the input features and the target variable.
4. **Prediction:** The trained linear regression model is used to make predictions (y_{pred_lr}) on the testing data (x_{test}).
5. **Evaluation:** Mean squared error (MSE) is calculated to assess the performance of the linear regression model on the testing data. Lower MSE indicates better performance.

Finally, the MSE value is printed to evaluate the performance of the linear regression model.

Neural Network implementation

```
# Define the model
nn_model = Sequential()
nn_model.add(Dense(128, input_dim=X_train.shape[1], activation='relu')) #This adds a dense
(fully connected) layer with 128 neurons and ReLU activation.
nn_model.add(Dense(64, activation='relu'))
nn_model.add(Dense(1))

# Compile the model
nn_model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model
nn_model.fit(X_train, y_train, epochs=20, batch_size=52, validation_split=0.1)

# Make predictions
y_pred_nn = nn_model.predict(X_test)

# Evaluate the model
mse_nn = mean_squared_error(y_test, y_pred_nn)
print(f'Neural Network MSE: {mse_nn}')
```

1. **Model Definition:** A neural network model (`nn_model`) is defined using the Keras Sequential API. It consists of three layers:
 - The first layer is a dense (fully connected) layer with 128 neurons and ReLU activation, taking input with dimensions equal to the number of features in the training data.
 - The second layer is another dense layer with 64 neurons and ReLU activation.

- The third layer is a single neuron, representing the output layer for regression tasks.
- 2. **Model Compilation:** The model is compiled using the Adam optimizer and mean squared error loss function.
- 3. **Model Training:** The model is trained on the training data (`x_train`, `y_train`) for 20 epochs with a batch size of 52. Additionally, a validation split of 0.1 is used to monitor the model's performance on a fraction of the training data during training.
- 4. **Prediction:** The trained neural network model is used to make predictions (`y_pred_nn`) on the testing data (`x_test`).
- 5. **Evaluation:** Mean squared error (MSE) is calculated to evaluate the performance of the neural network model on the testing data. Lower MSE indicates better performance.

3 Visualization

In this section, we visualize the review content using a word cloud and a correlation heatmap. The word cloud provides a visual representation of the most frequently occurring words in the reviews, while the heatmap shows the correlation between different features in the dataset.

3.1 Word Cloud Visualization

3.1.1 Word Cloud Explanation

A word cloud is a visual representation of text data, where the size of each word indicates its frequency or importance. In this project, we generate a word cloud from the review content to identify the most common words used by the reviewers.

3.1.2 Word Cloud Code

Below is the Python code used to generate the word cloud:

Listing 3: Word Cloud Generation

```
from wordcloud import WordCloud

# Generate a word cloud from the review content
text = " ".join(review for review in df['content']).astype(str))

#This generates a word cloud from the 'content' of the reviews.
wordcloud = WordCloud(width=800, height=400, background_color='white').generate(text)

# Display the word cloud
plt.figure(figsize=(10, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
```



```
plt.title('Word Cloud of Review Content')
plt.show()
```

3.1.1 Code Explanation

The code above performs the following steps:

1. Import the necessary libraries: WordCloud from the wordcloud package and matplotlib.pyplot from the matplotlib package.
2. Combine all review content into a single string of text.
3. Generate a word cloud from the combined text, specifying parameters such as width, height, and background color.
4. Display the generated word cloud using matplotlib.



3.2 Correlation Heatmap Visualization

3.2.1 Correlation Heatmap Explanation

A correlation heatmap is a graphical representation of the correlation matrix that shows the relationship between different features in the dataset. In this project, we create a heatmap to visualize the correlation between the review scores and the number of thumbs up each review received.

3.2.2 Correlation Heatmap Code

Below is the Python code used to generate the correlation heatmap:

Listing 4: Correlation Heatmap Generation

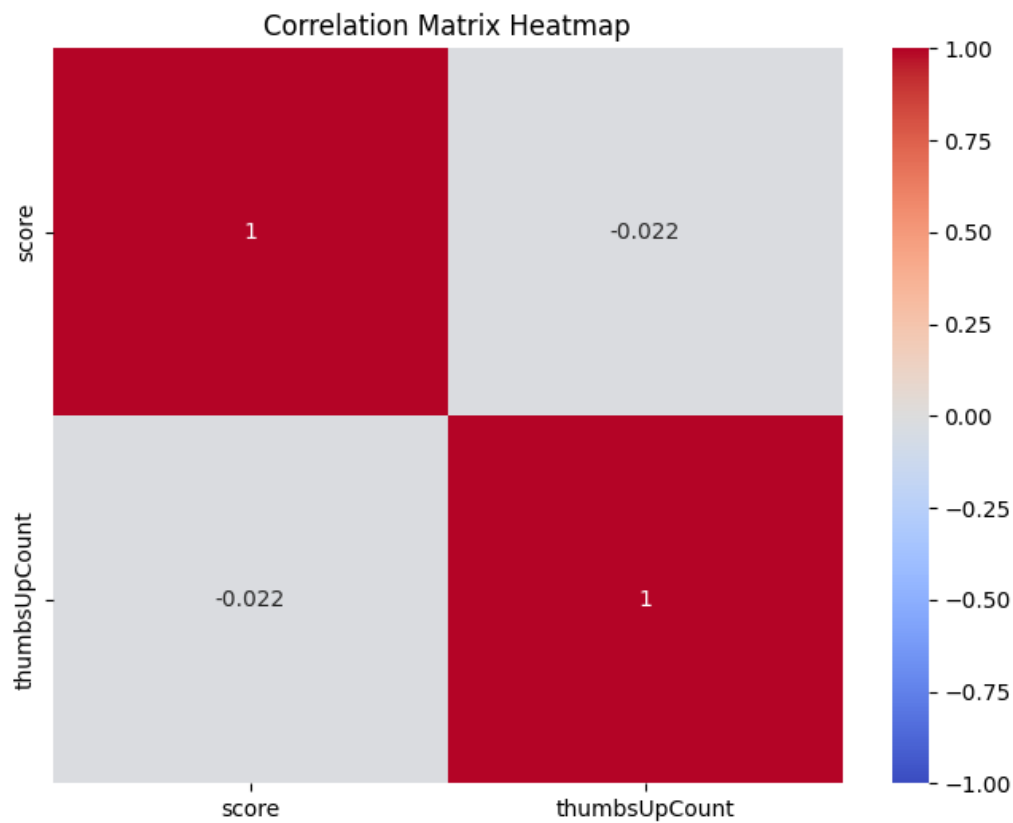
```
# Compute the correlation matrix
correlation_matrix = df[['score', 'thumbsUpCount']].corr()

# Generate a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1) #This creates
a heatmap of the correlation matrix.
plt.title('Correlation Matrix Heatmap')
plt.show()
```

3.2.3 Code Explanation

The code above performs the following steps:

1. Import the necessary libraries: `seaborn` and `matplotlib.pyplot`.
2. Compute the correlation matrix for the specified features ('score' and 'thumbsUpCount').
3. Generate a heatmap of the correlation matrix, specifying parameters such as figure size, color map, and value range.
4. Display the generated heatmap using `matplotlib`.



4 Experimentation and Results

4.1 Experimental Setup

The dataset is split into training and testing sets to evaluate the performance of the models. The data is standardized to ensure features have similar scales, improving model performance.

4.2 Evaluation Metrics

The models are evaluated using the following metrics:

- **Accuracy:** For classification models like logistic regression.
- **Mean Squared Error (MSE):** For regression models like linear regression and neural networks.
- **Confusion Matrix:** To evaluate the performance of classification models.

4.3 Results

The performance of the models is as follows:

- **Logistic Regression Accuracy:** 0.91
- **Classification Report of Logistics Regression**

Classification Report:				
	precision	recall	f1-score	support
0	0.76	0.41	0.53	3242
1	0.92	0.98	0.95	24003
accuracy			0.91	27245
macro avg	0.84	0.69	0.74	27245
weighted avg	0.90	0.91	0.90	27245

The classification report indicates that the model achieved high precision, recall, and F1-score for class 1, demonstrating its effectiveness in correctly identifying instances of this class. However, its performance for class 0 is relatively lower, suggesting room for improvement, particularly in correctly identifying instances of this class. Overall, the model achieved an accuracy of 91%, indicating its capability to correctly classify the majority of instances in the dataset.

- **Linear Regression MSE:** 0.7104640968073752
- **Linear Regression R2:** 0.3809818360539347
- **Neural Network MSE:** 0.743476045070499
- **Neural Network R2:** 0.3474131957994425

6.2 Discussion

This project helps us to understand the basics and techniques that are present in the machine learning. It analyzed user reviews of the ChatGPT

Android App using various machine upto a great extend. The logistic regression model was effective for classification tasks, while the neural network model provided a better performance for predicting scores. These insights can be valuable for improving user experience and app features based on user feedback.

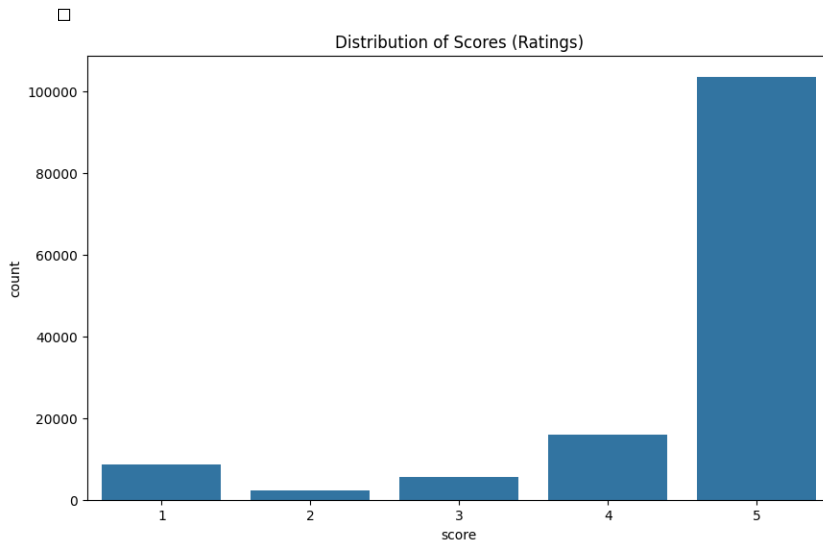
7 Conclusion

7.1 Conclusion

The analysis of ChatGPT reviews using machine learning techniques provided valuable insights into user sentiment and app performance. The visualization techniques, including word cloud and correlation heatmap, helped in understanding the data distribution and relationships between features. The logistic regression is most helpful, linear regression, and neural network models were little effective in classifying and predicting review scores. Future work can involve exploring more advanced models and techniques for better predictions and insights.

8 References

- [Kaggle dataset: ChatGPT Reviews - Daily Updated](#)
- [Pandas documentation: Pandas](#)
- [Matplotlib documentation: Matplotlib](#)
- [Seaborn documentation: Seaborn](#)
- [Scikit-learn documentation: Scikit-learn](#)
- [TensorFlow/Keras documentation: TensorFlow/Keras](#)
- [WordCloud documentation: WordCloud](#)
- [Numpy documentation: Numpy](#)



10 Future Scope

Future work can explore more advanced machine learning models, enhance data preprocessing techniques, and apply sentiment analysis to gain deeper insights into user feedback. Additionally, real-time analysis and automated feedback systems could be developed to provide instant insights and recommendations.

10.Appendix

10.1 Appendix A:Full code

https://colab.research.google.com/drive/1W7jcLr4-1YoVLp5nCGd3-nBv_M4UwsSY#scrollTo=34wfoxNY-xAP

Appendix B: Screenshots of Output

Figure 1: Distribution of Scores (Ratings)

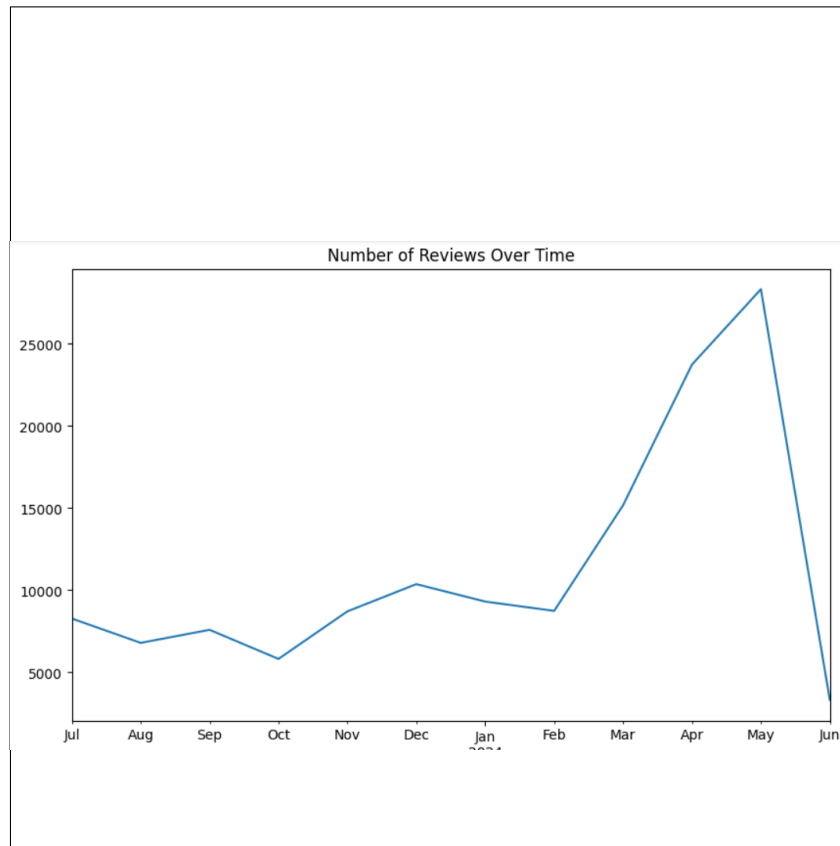


Figure 2: Number of Reviews Over Time

